RENSSELAER MECHATRONICS

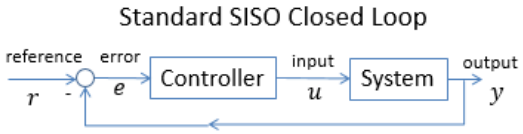
DC Motor Control with Full State Feedback

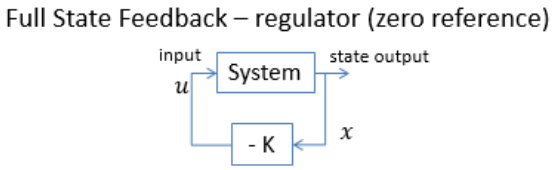
Objectives:

- Use Full State Feedback (FSF) to meet the same transient specifications as previous labs
- Address steady-state error with FSF
- Implement LQR FSF

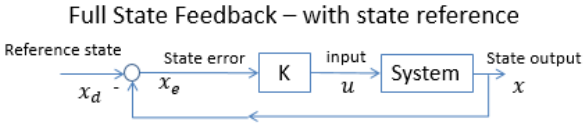
Background Information:

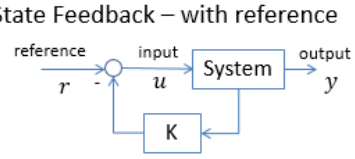
The standard SISO controller uses the output compared to a reference. A full state feedback compensator uses the complete state multiplied by a gain matrix as the control law $u = -Kx$. If the reference state is zero, this compensator is referred to as a regulator.





If the reference is not zero, the easiest way to add a reference is to add a desired reference state x_d. Another way is to add a reference signal the same size as the input, but this requires additional design considerations:





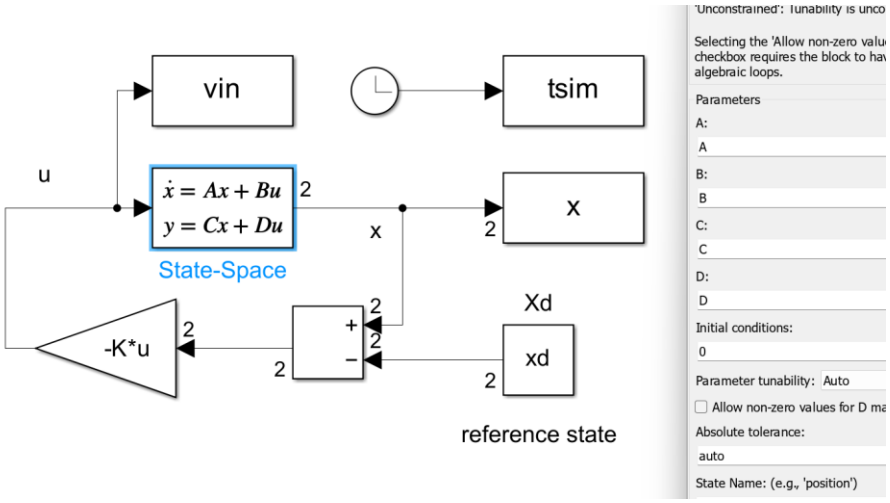
All of these different architectures have the same basic control law $u = -Kx$.

Full State Feedback:

Given the model transfer function: $\frac{\theta}{V} = \frac{K}{s(\tau s+1)}$ and state $x = \left[\theta\ \dot{\theta}\right]^T$, with $K = 5.5, \tau = 0.045$

- What are the state space matrices A, B? (do this by hand, if you use tf2ss you won't necessarily know what the states are).
- Using the 'place' command in MATLAB what is the gain matrix K required to meet the same transient specifications as the previous section: 5% overshoot and settling time of 0.25 seconds. (Refer to the previous lab to obtain the desired closed loop poles, or use SISOtool to find them)

Build this controller in Simulink:



This is a simulation diagram so you can use variable step ODE45 to solver for the system response. You will need to define state space matrices A and B, and the desired closed loop poles p1, and p2. Also be sure the gain is selected as Marix Multiplication.

```
%% Full State Feedback Simulatoin using place
K=place(A,B,[p1,p2])
xd=[1;0]; % use a reference of 1 so it is easy to find %OS

% simulate the step response and verify your controller performance
close all
tsim=.5;
C=eye(2,2);D=[0;0];
sim('DC_Motor_Position_SS_FSF.slx')
plot(tsim, x(:,1))
figure, plot(tsim,vin)
```

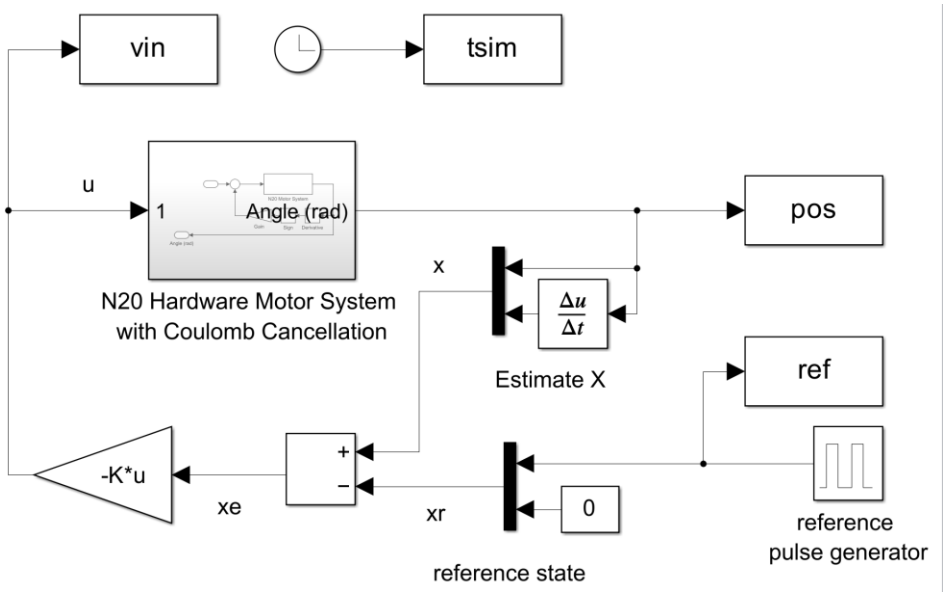
The control effort is still of the form $u = -Kx$. Notice the C matrix is identity so that the State-Space block has the full state as the output $x = \left[\theta\ \dot{\theta}\right]^T$.

- Plot the step response for a unit step
- What is the actual percent overshoot? If it is different than the designed overshoot, explain why.
- Plot the control effort. What is the magnitude of the control effort initially?

- Write out the control law of your resulting compensator (multiply everything out):
  - $u =$
- What can you say about the magnitude of the control effort initially (based off the above control law)? Why is this different from the previous lab?
- Compare the control laws from this section and the previous lab – what can you say about what type of compensator a FSF is?
- If you have finite control effort of 4.65 volts what is the largest step change in the reference position possible to avoid saturation (in radians)?

Implementation on Hardware

- Modify the previous Full State feedback simulation so it can be deployed to the hardware with the USB as the voltage supply.
- Use the hardware diagrams from the previous lab it will already have most of the blocks and parameters to run on the hardware:
  - Fixed step discreet solver with sample time of 0.005 seconds, stop time of 8 seconds
  - Use the motor hardware system that has coulomb friction cancellation
  - Use the pulse generator block to construct the reference state



Use the following code to compare the simulation results to the predicted results based on the model

```
%% Hardware:
% coulomb friction voltage estimate
% may want to adjust this higher for FSF
fb_c=0.4;
rdes=0.7;
```

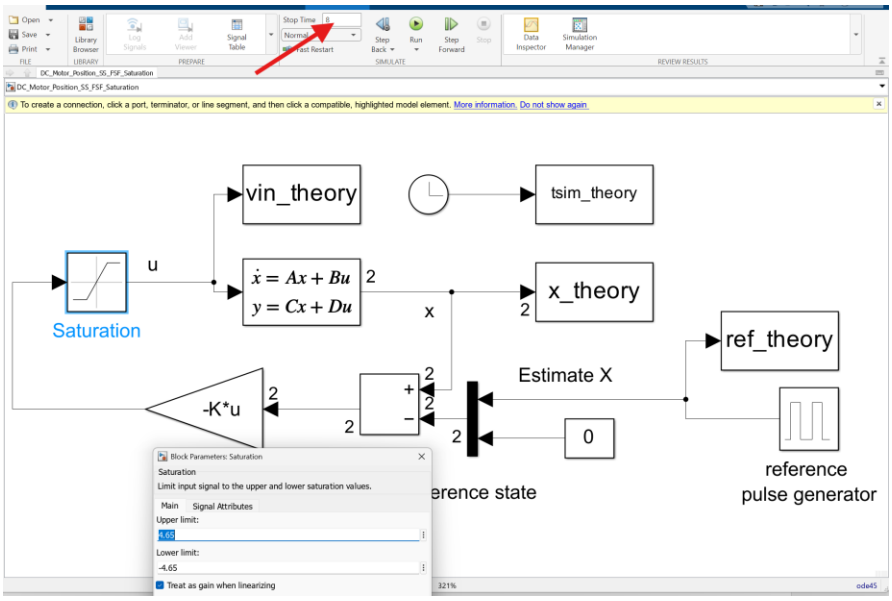
```
%% Open the simulink diagram and run experiment:

close all
plot(tsim, pos), hold on
plot(tsim, ref','r-')

% compare to simulation
Acl=A-B*K;Bcl=B*K(1); Ccl=[1 0]; Dcl=0;  Gcl_ss=ss(Acl,Bcl,Ccl,Dcl);
[y t]=lsim(Gcl_ss,ref,tsim);
plot(t,y, 'k--')
legend('experiment','referenece','theory')
```

Questions:

- Plot the response using the sample code above. These results should be similar to the results at the end of the previous lab. The response should match well but will have steady state error. You may have to increase/decrease the coulomb friction value to get a better match similar to how it was adjusted in the last lab – increase it until there are oscillations.

- Change the reference to pi/2. Run the experiment again and plot the results.

- The pi/2 test might not match well anymore because the large reference will cause saturation. Add a saturation block to the *model simulation* before the voltage input to capture the fact that the system only has 4.65 volts available. Change the stop time to 8 seconds and use the pulse input from the hardware experiment so the input is the same:



Include these theoretical results on the plot of the hardware. It should match better now that the model included the saturation.